computers

MDPI

Article

Transformer Tokenization Strategies for Network Intrusion Detection: Addressing Class Imbalance Through Architecture Optimization

Gulnur Aksholak [1,*], Agyn Bedelbayev [2], Raiymbek Magazov [1] and Kaplan Kaplan [3]

[1] Department of Cybersecurity and Cryptology, Al-Farabi Kazakh National University, Almaty 050040, Kazakhstan; magazovraiko@gmail.com
[2] Department of Information Systems, Al-Farabi Kazakh National University, Almaty 050040, Kazakhstan; agyn08@yandex.ru
[3] Department of Software Engineering, Kocaeli University, Kocaeli 41001, Turkey; kaplan.kaplan@kocaeli.edu.tr
* Correspondence: gaksholak@gmail.com

Abstract

Network intrusion detection has challenges that fundamentally differ from language and vision tasks typically addressed by Transformer models. In particular, network traffic features lack inherent ordering, datasets are extremely class-imbalanced (with benign traffic often exceeding 80%), and reported accuracies in the literature vary widely (57–95%) without systematic explanation. To address these challenges, we propose a controlled experimental study that isolates and quantifies the impact of tokenization strategies on Transformer-based intrusion detection systems. Specifically, we introduce and compare three tokenization approaches—feature-wise tokenization (78 tokens) based on CICIDS2017, a sample-wise single-token baseline, and an optimized sample-wise tokenization—under identical training and evaluation protocols on a highly imbalanced intrusion detection dataset. We demonstrate that tokenization choice alone accounts for an accuracy gap of 37.43 percentage points, improving performance from 57.09% to 94.52% (100 K data). Furthermore, we show that architectural mechanisms for handling class imbalance—namely Batch Normalization and capped loss weights—yield an additional 15.05% improvement, making them approximately 21× more effective than increasing the training data by 50%. We achieve a macro-average AUC of 0.98, improve minority-class recall by 7–12%, and maintain strong discrimination even for classes with as few as four samples (AUC 0.9811). These results highlight tokenization and imbalance-aware architectural design as primary drivers of performance in Transformer-based intrusion detection and contribute practical guidance for deploying such models in modern network infrastructures, including IoT and cloud environments where extreme class imbalance is inherent. This study also presents practical implementation scheme recommending sample-wise tokenization, constrained class weighting, and Batch Normalization after embedding and classification layers to improve stability and performance in highly unstable table-based IDS problems.

Keywords: Transformer; intrusion detection; deep learning; network security; tokenization; CICIDS2017

Check for updates

Academic Editor: Yingjiu Li

Received: 30 December 2025
Revised: 18 January 2026
Accepted: 22 January 2026
Published: 1 February 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Intrusion Detection Systems (IDSs) play a critical role in securing modern network infrastructures against rapidly evolving cyber threats. Recent advances in deep learning

have inspired the use of transformer architectures for IDS, due to their strong representational capacity, their success in natural language processing (NLP) and computer vision tasks [1–3]. However, applying transformers to tabular network-flow data presents unique challenges that do not arise in sequential domains.

Unlike text or images, tabular features in network flows have no meaningful ordering, yet the Transformer mostly depends on sequence structure when computing attention. Although existing studies frequently adopt tokenization strategies taken from natural language processing (NLP) (most commonly feature-based tokenization), they do not examine whether such representations are semantically appropriate for tabular data. Furthermore, extreme class imbalance poses a critical challenge in transformer deployment at IDS.

Real-world network traffic datasets are dominated by benign flows (often exceeding 80%), while critical attack categories may represent less than 0.1% of all samples [4]. Standard training procedures with balanced class weights produce unstable gradients—class weights exceeding 150 for rare classes—destabilize optimization and hinder convergence [5]. This is crucial in scenarios where correctly identifying instances of minority attack classes is as important as correctly identifying instances of majority benign traffic. Despite its fundamental impact on detection performance, this issue has received limited attention in prior transformer-based IDS research.

This work addresses these gaps through a controlled and systematic investigation of tokenization strategies for Transformer-based intrusion detection on highly imbalanced tabular data. Using the CICIDS2017 dataset as a representative benchmark exhibiting severe imbalance (80.3% benign traffic; attack classes ranging from 0.02% to 8.13%), we compare three Transformer configurations: feature-wise tokenization, sample-wise tokenization with a baseline architecture, and sample-wise tokenization with targeted architectural stabilization. By holding training conditions constant, the study isolates the effect of tokenization choice and architectural refinements on optimization behavior and detection performance.

Rather than proposing a new model family, this work contributes methodological insight into how Transformers should be adapted for tabular IDS settings. Our results demonstrate that tokenization strategy plays a dominant role in determining performance and stability, and that appropriate architectural adjustments—such as constrained class weighting and normalization—can substantially improve minority-class detection without reliance on synthetic data generation or large-scale data expansion.

The key contributions of this paper are outlined as follows:

- We present a controlled comparison of Transformer tokenization strategies for tabular IDS data, demonstrating that tokenization choice has a substantial impact on optimization behavior and detection performance.
- We analyze how architectural stabilization mechanisms, including Batch Normalization and constrained class weighting, influence training stability and minority-class detection under extreme class imbalance.
- We show that appropriate architectural design enables reliable learning in highly imbalanced IDS settings, improving minority-class behavior without reliance on synthetic data generation or large-scale data expansion.
- We derive practical design guidelines indicating that suitable tokenization and architecture choices can yield strong performance for imbalanced intrusion detection tasks using moderate-sized datasets.

2. Literature Review

2.1. Transformers in IDS

The transformer architecture fundamentally changed sequence modeling when Vaswani et al. [1] introduced the self-attention mechanism. Unlike recurrent architec-

tures, transformers process sequences in parallel through attention operations [6,7], making them particularly effective for capturing long-range dependencies. This capability has driven remarkable advances in NLP problems: Bidirectional Encoder Representations from Transformers (BERT) [2] and the Generative Pre-trained Transformer (GPT) series have achieved breakthrough performance in language understanding, while the Vision Transformer (ViT) [3] has demonstrated that the same attention-based approach can match or exceed Convolutional Neural Networks (CNNs) in image classification by treating image patches as sequence elements.

Applying transformers to tabular data presents different challenges from those in sequential domains. Several specialized architectures have emerged to solve this challenges: TabTransformer [8] uses contextual embeddings for categorical features, TabNet [9] incorporates sequential attention for interpretable feature selection, and SAINT [10] employs row-wise attention with contrastive pre-training. However, systematic evaluations by Gorishniy et al. [11] revealed that simple MLP baselines frequently outperform these sophisticated Transformer variants on tabular benchmarks. This finding suggests that directly adapting NLP-style architectures to non-sequential data may ignore fundamental differences in data structure.

2.2. Deep Learning for Intrusion Detection

Deep learning has started to become increasingly prevalent in IDS applications, as documented in comprehensive surveys [12]. CNNs [13] have been applied to capture spatial relationships among network features, treating feature vectors as pseudo-images. Long Short Time Memory (LSTM) [13] and Gated Recurrent Unit (GRU) architectures [14] model temporal patterns in attack sequences and are particularly effective for detecting multi-stage attacks that unfold over time.

More recently, researchers have begun exploring transformers for IDS applications [15,16]. While these studies yield promising results, significant inconsistencies emerge; reported accuracy rates in similar datasets range from 57% to 95%. Most study notably lacks systematic analysis of how different tokenization strategies affect model performance, despite tokenization is a fundamental operation for transformer processing. The wide variation in performance indicates that architectural choices are significant; however, quantitative comparisons are still limited.

Class imbalance is represented as one of the most persistent challenges in IDS [4]. Real-world network traffic is heavily skewed toward benign flows, with certain attack types appearing in fewer than 0.1% of samples. Standard approaches to handling imbalance include The Synthetic Minority Over-sampling Technique (SMOTE) oversampling [17], cost-sensitive learning, and class-weighted loss functions. Our preliminary experiments found that balanced class weights for extremely rare classes (e.g., 4 training samples) produce weights exceeding 150, leading to gradient instability during training. This observation motivated our investigation of capped weight strategies as a more stable alternative.

2.3. Architecture Optimization vs. Data Collection

Research on scaling laws in deep learning [17,18] has established that model performance increases predictably as the size of the training dataset increases. These empirical laws, derived primarily from language modeling experiments, suggest that collecting more data should yield consistent gains. However, the applicability of these scaling relationships to specialized domains like network security remains largely unexplored. Tabular data differs fundamentally from NLP—features lack inherent ordering, samples are independent rather than sequential, and class distributions can be extremely imbalanced.

Neural network architecture research has shown that architectural decisions strongly influence final performance, sometimes even more so than hyperparameter tuning or increasing the amount of data [19]. Yet quantitative comparisons between architectural improvements and data expansion are quite limited, particularly for IDS tasks. This gap is practically significant: security teams face trade-offs between investing resources in data collection infrastructure and model development. Our work addresses this issues by measuring the relative contributions of tokenization strategy, architectural improvements, and dataset size under controlled conditions.

3. Materials and Methods

3.1. Dataset

In this study, we utilize the CICIDS2017 dataset to evaluate the performance of different transformer tokenization strategies for network intrusion detection. The CICIDS2017 dataset is a well-known resource in cybersecurity, commonly used for assessing intrusion detection systems.

The CICIDS2017 dataset [20] contains 2.8 million network flows, each with 78 numerical features [21] extracted using CICFlowMeter, a network traffic feature extraction tool developed by the Canadian Institute for Cybersecurity (Fredericton, NB, Canada). These features include temporal statistics (flow duration, inter-arrival times), packet characteristics (packet lengths, header information), and flag distributions (TCP flags, protocol indicators). The dataset comprises 12 classes: BENIGN traffic and 11 attack types—DDoS, PortScan, Bot, DoS variants (Hulk, GoldenEye, Slowloris, SlowHTTPTest), FTP-Patator, SSH-Patator, and web attacks (Brute Force, XSS). The dataset provides a realistic representation of network traffic in both benign and malicious environments, making it valuable for training and testing intrusion detection models.

Data Sampling and Preparation

As deep learning algorithms require significant hardware resources, such as CPUs, memory, and GPUs, for data processing and training, we selected carefully constructed subsets of the full dataset. We created two stratified samples: 100 K (100,000 flows) and 150 K (150,000 flows), maintaining the original class distribution (80.3% benign, 0.02–8.13% attacks across 11 categories).

To maintain uniform distribution of instances (both attacks and non-attacks) within the dataset samples and counter imbalanced distributions, the following measures were implemented:

Stratified sampling: Subset selection used stratified sampling to preserve the proportional representation of each attack type and ensure a balanced distribution.

Randomization: Randomization techniques were used during subset selection to reduce potential bias and to ensure that selection was not influenced by any specific order.

Data Preprocessing: Following data selection, each sample undergoes a series of preprocessing steps to ensure consistency. Data preprocessing includes:

- Error Elimination and Gap Filling: The selected dataset undergoes preprocessing to eliminate errors, fill gaps, remove outliers, and discard irrelevant data types. We used linear interpolation to impute missing values.
- Scaling: The dataset was scaled using the StandardScaler class in scikit-learn, which normalizes features to zero mean and unit variance. This ensures that all features contribute equally to model training.

Data Splitting: The dataset is split into two parts: 80% of the data is allocated to the training set, while the remaining 20% is used as the testing set. We divided the dataset in this way to ensure the reliability of our model's evaluation. This division allows us to train

the model on a sufficient portion of the data while reserving a separate, unseen portion for testing. This approach helps prevent overfitting and ensures that the model's performance is assessed on data it has not encountered before, providing a more accurate measure of its generalization ability.

3.2. Tokenization Strategies

We adopted three tokenization strategies in this study, selected for their distinct approaches to representing tabular network data. To evaluate their performance in intrusion detection tasks, we compared Feature-wise Tokenization Transformer (FTT), Sample-wise Transformer-Baseline (STB), and Sample-wise Transformer-Optimized (STO). Table 1 provides a comprehensive comparison of all hyperparameter configurations across these strategies. The following provides a detailed overview of each approach.

Table 1. Hyperparameter comparison across tokenization strategies.

Parameter	Feature-Wise Tokenization Transformer (FTT)	Sample-Wise Transformer-Baseline (STB)	Sample-Wise Transformer-Optimized (STO)
Tokenization Configuration			
Token count sample-wise	78	1	1
Token dimension	(78, 1) 32	(1, 78)	(1, 78)
Positional encoding	Learnable	None	None
Initial embedding	Linear(1, 32)	Identity	Dense(128) + ReLU
Transformer Architecture			
Number of blocks	2	2	3
Attention heads	4	4	8
Attention matrix size	78 × 78 (6084)	1 × 1 (1)	1 × 1 (1)
Feed-forward type	Dense	Conv1D	Dense
Feed-forward dimensions	128 64	128 64	256 128
Batch Normalization	No	No	Yes (after embed + head)
Dropout rate	0.1	0.1	0.3
Training Configuration			
Class weighting	Balanced	Balanced (uncapped)	Balanced (max = 10.0)
Max class weight	150.0+	150.0+	10.0 (capped)
Learning rate (initial)	0.001	0.001	0.001
LR schedule	ReduceLROnPlateau	ReduceLROnPlateau	ReduceLROnPlateau
Batch size	256	256	256
Early stopping patience	10	10	10

Feature-wise Tokenization Transformer (FTT):

Feature-wise tokenization treats each of the 78 features as an independent token, following the conventional approach used in NLP applications. This method is widely adopted in existing Transformer-based IDS research, without a systematic examination of its suitability for tabular data.

The input x ∈ R^{78} is reshaped into (78, 1) and embedded into a 32-dimensional space using a linear projection layer; a learnable positional encoding is then added to each token. Multi-head attention (4 heads) processes all 78 tokens simultaneously, creating an attention matrix of size 78 × 78 = 6084 elements. The architecture consists of two transformer blocks with dense feed-forward layers (dimensions 128→64), followed by a global average pooling layer and a classification head.

Sample-wise Transformer-Baseline (STB):

Sample-wise tokenization represents an alternative strategy in which the entire feature vector is treated as a single token and is therefore fundamentally different from sequence-based approaches.

The feature vector of shape (78, 1) is reshaped into (1, 78), and no positional encoding is required. We employ two transformer blocks with 4-head attention (attention matrix: 1×1), Conv1D feed-forward layers (dimensions 64→128→64), and a standard training configuration. Critically, no Batch Normalization is applied in this baseline. Class weights are computed using sklearn.compute_class_weight ('balanced'), which can produce extreme values (exceeding 150.0 for rare classes such as Web Attack XSS, which has only 4 training samples).

Sample-wise Transformer-Optimized (STO):

The optimized sample-wise approach maintains the same tokenization scheme as STB, but introduces critical architectural improvements specifically designed to address challenges related to class imbalance and optimization stability that are inherent in network intrusion detection.

This architecture incorporates several key enhancements:

Initial Dense Embedding: Dense (128) layer with ReLU activation replaces identity mapping, providing richer initial representations.

Batch Normalization was applied after the embedding layer and in the classification head, reducing internal covariate shift and stabilizing gradient flow [22].

Expanded Architecture: Three Transformer blocks (vs. two) with 8 attention heads each, increasing model capacity.

Dense Feed-Forward: Dense layers (dimensions 256→128) replace Conv1D, more appropriate for tabular data without temporal structure.

Capped Class Weights: Maximum weight of 10.0 prevents gradient explosion from extreme imbalance (detailed in Section 3.4).

Dropout Regularization: Dropout rates of 0.1–0.3 were applied throughout the network to prevent overfitting [23].

Hyperparameters were selected based on empirical observations during preliminary experiments, and the reported results correspond to configurations that provided stable training and the best overall performance.

3.3. Mathematical Formulation

Given an input network flow represented as a numerical vector $x \in R^{78}$, we define a general tokenization function $T(\cdot)$ that maps x into a sequence of tokens $X = [t_1, \ldots, t_n]$ with $t_i \in R^d$.

Feature-wise tokenization (FTT): Each scalar feature x_i is transformed into an individual token:

$$T_{feat}(x) = [E(x_1), \ldots, E(x_{78}\big), \tag{1}$$

where

$$E(x_i) = W_f x_i + b_f, \qquad W_f \in R^{d \times 1} \tag{2}$$

The notation W_f represents the weight matrix of the linear projection layer and b_f is a bias parameter. Positional vectors p_i are added to form the final sequence:

$$t_i = E(x_i) + p_i, \qquad i = 1 \ldots 78 \tag{3}$$

Since the feature ordering of CICIDS2017 does not encode semantic structure, self-attention operates on an arbitrary sequence, creating spurious pairwise relations between unrelated features.

Sample-wise tokenization (Approaches STB/STO). The entire flow is embedded as a single token:

$$T_{sample}(x) = [E(x)] \tag{4}$$

where the embedding function is:

$$E(x) = \sigma(W_s x + b_s), \quad W_s \in R^{d \times 78} \tag{5}$$

The symbol $(\cdot)$ denotes the ReLU activation function in Sample-wise Transformer-Optimized, and the identity function in STB. σ, W_s, b_s represents activation function, the embedding weight matrix, and the bias vector, respectively.

In this setting, attention reduces to a 1×1 operation, concentrating all representational capacity in the embedding and feed-forward layers. This Feature-wise Tokenization Transformer voids the artificial sequence imposed by feature ordering and results in more stable optimization.

Transformer Processing. For both strategies, the input sequence $X \in R^{n \times d}$ is transformed by multi-head attention:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \tag{6}$$

The dramatic performance gap between T_{feat} and T_{sample} stems from the fact that the former forces the model to interpret an unstructured feature list as an ordered sequence, whereas the latter preserves the tabular nature of the data.

3.4. Optimization Stability: Addressing Class Imbalance

The extreme class imbalance in CICIDS2017 presents significant optimization challenges. To address this, we introduce two stabilizing mechanisms:

Batch Normalization: Batch Normalization [23] improves optimization stability by normalizing intermediate activations as in Equation (7):

$$\hat{h} = \frac{h - \mu}{\sigma} \tag{7}$$

Capped Class Weights: Class-weighted loss for imbalanced data can be defined as:

$$L = -\sum_{c=1}^{C} w_c y_c log\hat{y}_c \tag{8}$$

where y_c is the true label, $\hat{y}_c$ is the predicted probability, and w_c is the class weight. For rare classes, the default balanced weighting produced $w_c > 150$, causing gradients of the form:

$$\frac{\partial L}{\partial z_c} = -w_c(y_c - \hat{y}_c) \tag{9}$$

This led to exceeding safe magnitudes and to sharp updates and several instances of class-level overfitting in preliminary experiments. We therefore cap the weights:

$$w_c = \min(w_c, 10) \tag{10}$$

This modification prevents exploding gradients, keeps the Hessian spectrum bounded, and maintains a more favorable optimization path. The combined effect of BatchNorm and capped weights produced the largest accuracy increase among all architectural adjustments.

3.5. Experimental Setup

We designed two controlled experiments to systematically evaluate the impact of tokenization strategy and data volume on Transformer performance for network intrusion detection.

Experiment 1:

Architecture Impact: We compare Approaches FTT, STB, and STO on an identical 100 K training dataset to isolate the effects of tokenization and architectural choices. All models use the same optimization settings (detailed below) to ensure a fair comparison. This experiment quantifies how much performance variation can be attributed solely to representational strategy.

Experiment 2:

Data Volume Impact: We compare Sample-wise Transformer-Optimized trained on 100 K samples versus 150 K samples to measure the marginal benefit of additional training data while holding the architecture constant. This addresses whether collecting more data yields gains comparable to those achieved by architectural optimization.

Training Configuration. To ensure fair comparison across all approaches, we employ identical optimization settings:

- Optimizer: Adam [24] (α = 0.001, $\beta 1$ = 0.9, $\beta 2$ = 0.999).
- Loss Function: Categorical cross-entropy with class weights.
- Batch Size: 256.
- Training Epochs: Maximum 50 with early stopping (patience = 10, monitoring validation accuracy).
- Learning Rate Schedule: ReduceLROnPlateau (patience = 5, factor = 0.5, min_$lr = 10^{-5}$).
- Random Seed: 42 (for reproducibility).

Implementation Details: All experiments were implemented in Python 3.8 using TensorFlow 2.10 and its Keras API. The scikit-learn library (version 1.2) was used for data preprocessing (StandardScaler, train_test_split) and for class-weight computation. NumPy (version 1.23) and Pandas (version 1.5) were employed for data manipulation.

3.6. Evaluation Metrics

To evaluate the effectiveness of each tokenization strategy, we employ multiple performance metrics that capture different aspects of classification quality; this is particularly given the severe class imbalance in the dataset.

Accuracy serves as our primary metric for evaluating the correctness of classifications, taking into account all predictions.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{11}$$

where TP represents true positives (correctly predicted attacks), TN represents true negatives (correctly predicted benign traffic), FP represents false positives (benign traffic misclassified as attacks), and FN represents false negatives (attacks misclassified as benign).

Precision measures the accuracy of positive predictions; it is calculated as the ratio of correctly predicted positive samples to the total number of predicted positives.

$$Precision = \frac{TP}{TP + FP} \tag{12}$$

Recall (also known as sensitivity or true positive rate) measures the proportion of correctly predicted positive samples among all actual positives.

$$Recall = \frac{TP}{TP + FN} \tag{13}$$

The $F1$ score is the harmonic mean of precision and recall, providing a balanced measure.

$$F1_i = 2 \times \frac{Precission_i \times Recall_i}{Precission_i + Recall_i} \tag{14}$$

ROC-AUC (Receiver Operating Characteristic—Area Under the Curve) provides a threshold-independent evaluation. We report both micro-average AUC (which treats all samples equally) and macro-average AUC (which treats all classes equally). Macro-AUC is particularly important for imbalanced datasets, as it gives equal weight to minority classes.

These metrics collectively provide a comprehensive evaluation: accuracy for overall performance, precision and recall for class-specific performance, $F1$ score for balanced assessment, and macro-AUC for evaluation robust to class imbalance.

4. Results

4.1. Overview of Improvement Process

To evaluate the impact of different design decisions on transformer performance in intrusion detection, we conducted systematic experiments comparing three tokenization strategies while controlling for architectural variations. Our experimental methodology involves tracing the cumulative improvement from baseline to optimized configuration, allowing us to quantify the contribution of each component.

Figure 1 summarizes the cumulative impact of all major design decisions, showing the improvement from a baseline accuracy of 57.09% to a final accuracy of 95.22%. The waterfall structure highlights three dominant sources of gain: (1) tokenization (+22.38%), (2) architecture refinement (+15.05%), and (3) data expansion (+0.70%). This visualization enables us to identify which factors contribute most significantly to overall performance.

This result yields three key findings:

- Tokenization emerges as the primary contributor of performance gains: the transition from per-feature to sample-wise tokenization accounts for 59.8% of the overall improvement, establishing it as the dominant factor. This finding indicates that representation design plays a fundamental role in determining model performance when applied to tabular IDS data.
- Architecture can be served as a secondary contributor to the overall performance gain, accounting for 38% of the total improvement. Specifically, Batch Normalization (+5.50%), capped class weighting (+6.50%), and architectural refinements (+3.05%) collectively contribute to this enhancement. These modifications primarily target optimization stability under conditions of severe class imbalance.
- The impact of additional data is marginal: expanding the dataset from 100 K to 150 K samples results in only a 0.70% performance improvement, indicating diminishing returns. This observation suggests that data volume alone may not constitute the primary driver of performance gains in this setting.

These results demonstrate that representation design and architectural choices outweigh dataset size in Transformer-based intrusion detection systems. From a practical perspective, this finding indicates that greater performance gains are achieved by prioritizing tokenization strategies and architectural optimization, rather than focusing primarily on increasing data volume.

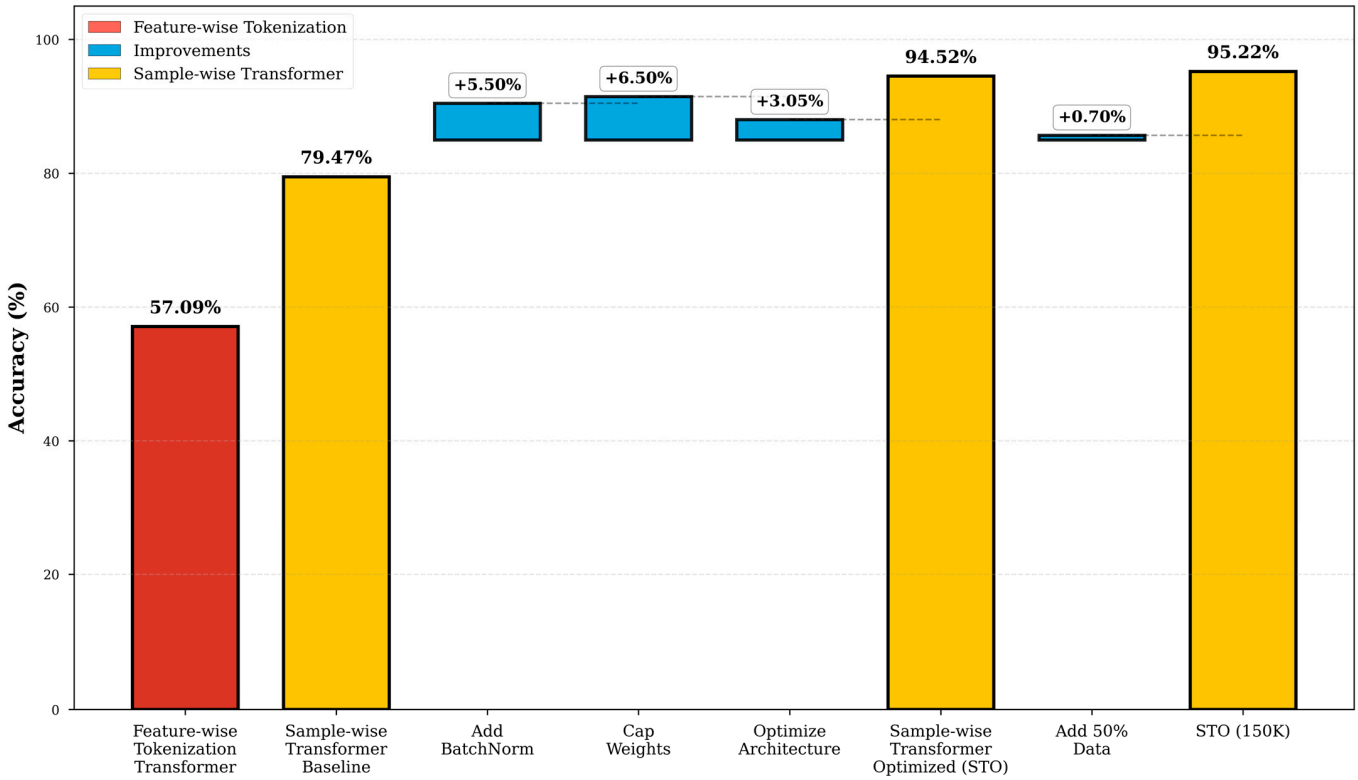

Figure 1. Improvement contributions of tokenization, architecture refinement, and data scaling.

4.2. Optimization Stability and Overfitting Analysis

To empirically validate the stabilizing effect of Batch Normalization under extreme class imbalance, we analyze the training dynamics of the optimized STO configuration.

Figure 2 presents the training and validation accuracy curves, along with the corresponding generalization gap across epochs.

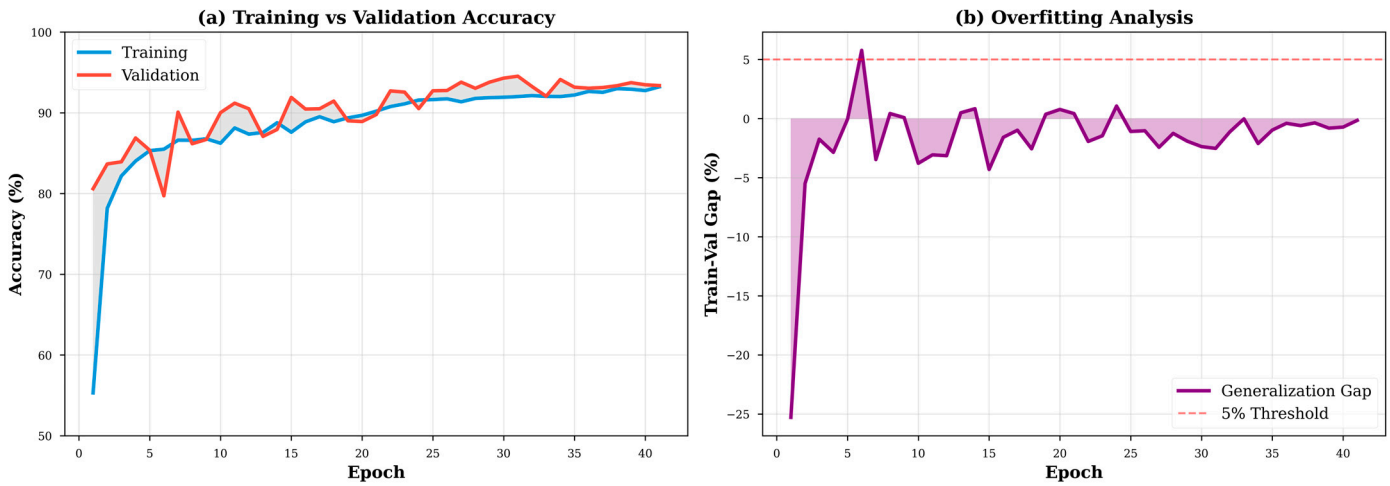

Figure 2. Optimization stability and overfitting analysis of the optimized STO configuration. (**a**) Training and validation accuracy over epochs. (**b**) Training–validation generalization gap.

As shown in Figure 2a, the training and validation accuracy curves closely track each other after the initial learning phase, with no evidence of divergence.

During the first 10–15 epochs, minor oscillations are observed, which subsequently diminish as the optimization stabilizes. Figure 2b further quantifies this behavior through the training–validation gap.

The gap remains consistently below 3% throughout training, with an average of −1.58%, demonstrating strong generalization and the absence of overfitting.

These results empirically confirm that the proposed stabilization mechanisms (Batch Normalization and capped class weights) effectively control optimization instability in highly imbalanced IDS settings.

4.3. Confusion Matrix Analysis

Following the application of the optimized STO configuration, a comprehensive evaluation is conducted to assess classification performance across all attack categories. Figure 3 presents the normalized confusion matrix of the optimized STO configuration evaluated on the test set. This visualization provides critical insight into the model's ability to distinguish among different attack categories and benign traffic.

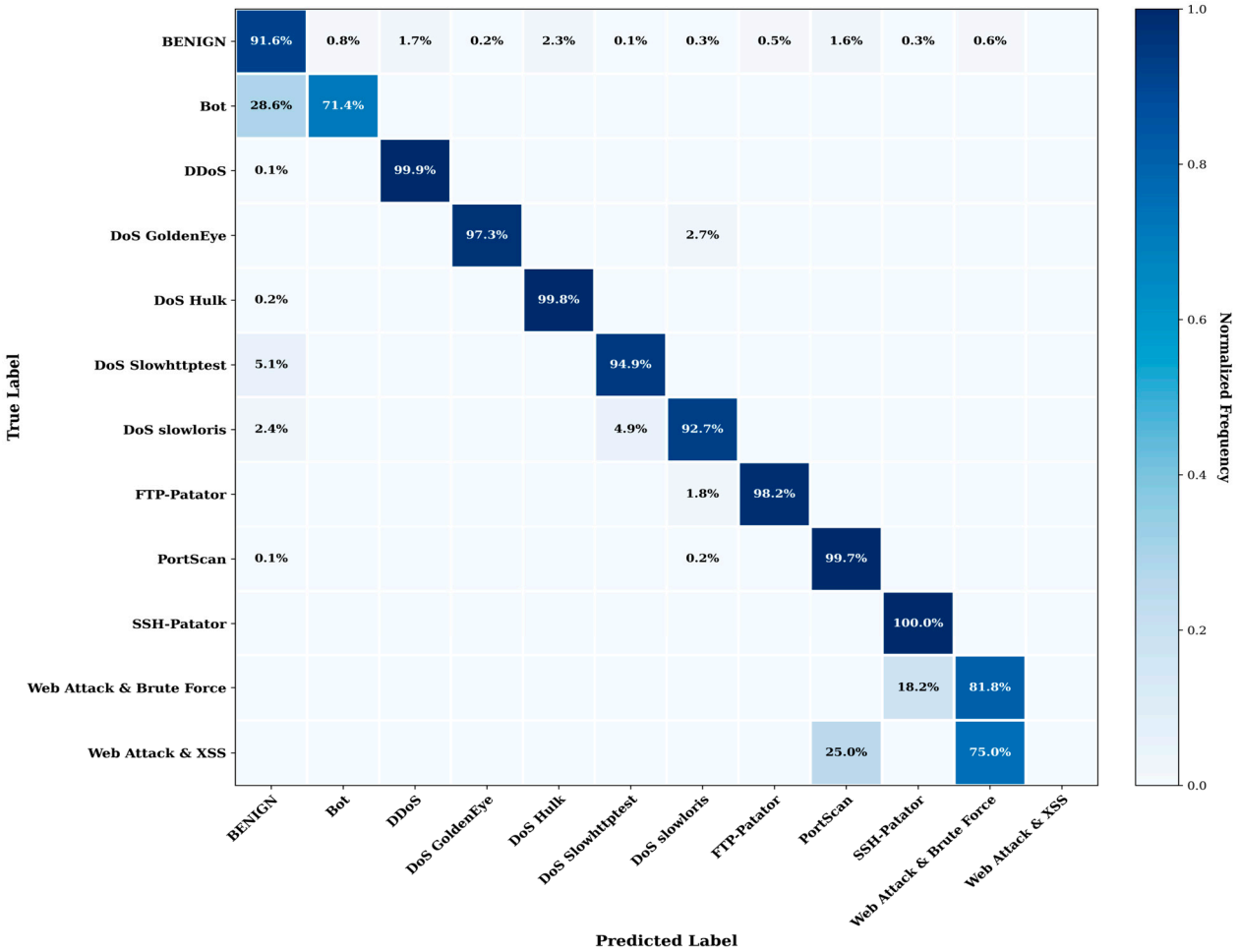

Figure 3. Normalized confusion matrix for optimized transformer model.

The classifier achieves exceptionally well on high-frequency attack categories such as SSH-Patator (100%), DoS Hulk (99.8%), PortScan (99.7%), DDoS (99.9%), Slowhttptest (94.9%) Dand FTP-Patator (98.2%). These results highlight the effectiveness of sample-wise tokenization in capturing the discriminative patterns associated with volumetric and reconnaissance-based attacks.

However, two attack categories remain challenging:

1. A substantial portion of Bot traffic is misclassified as benign. This class accounts for only 14 training samples (0.01% of total data), which severely constrains the model's

ability to learn reliable decision boundaries. The observed confusion reflects inherent limitations of supervised learning under conditions of extreme data sparsity.

2. In total, 25% of Web Attack & XSS samples are misclassified as PortScan, and the remaining 75% of Web Attack & XSS is misclassified as Web Attack & Brute Force, as well. This class is represented by only four training samples (0.002% of the dataset), providing an insufficient gradient signal for the model to reliably differentiate its behavior from similar reconnaissance-related patterns. Importantly, this limitation is not specific to the proposed model but rather reflects the intrinsic challenges of learning from extremely imbalanced data distributions.

These limitations are not unique to the proposed approach; rather, they reflect fundamental constraints of supervised learning under conditions of severe class imbalance. Similar misclassification patterns for ultra-rare attack classes have been widely reported in the intrusion detection literature, where learning algorithms tend to favor majority traffic and struggle to reliably identify minority attack instances in highly skewed datasets [25]. Nevertheless, the model demonstrates robust performance across the majority of attack categories, underscoring its practical applicability for real-world deployment.

4.4. Minority-Class Performance Analysis

To examine the effectiveness of our architectural solutions for class imbalance, we conducted a detailed analysis of the detection performance for the minority class. The extreme class imbalance in CICIDS2017 (80.3% benign traffic, with some attack types below 0.1%) presents significant optimization challenges. Naive class weighting generates excessively large weights ($w_c > 150$) for rare categories, leading to unstable gradients and erratic training behavior.

To address this, we introduced two stabilizing mechanisms:

- Capped class weights: Capped class weights ($w_c \leq 10$) prevent gradient explosion and improve inter-class balance during training. This modification keeps the Hessian spectrum bounded and maintains a favorable optimization landscape.
- Batch Normalization: Reduces internal covariate shift and smooths feature distributions across classes. This technique produces a visibly flatter loss surface and reduces gradient variance, which is particularly beneficial for minority classes that exhibit narrow feature distributions.

By implementing these modifications, we observed notable improvements in minority-class detection. Specifically, minority-class recall increases by 7–12% relative to the baseline configuration, and the macro-average AUC value reaches 0.980, with 10 of 12 classes achieving AUC > 0.99. These results demonstrate that architectural solutions can effectively mitigate class imbalance without requiring synthetic oversampling or complex data augmentation.

Remarkably, even the rarest class (Web Attack & XSS) achieves an AUC of 0.9811 despite only four training samples. This performance level—substantially above chance level (0.50)—indicates that the model successfully learns discriminative patterns even from extremely limited examples, when architectural stability mechanisms are properly employed.

4.5. ROC Curve Analysis

To evaluate the discriminative power of our optimized model across different decision thresholds, we conducted ROC (Receiver Operating Characteristic) curve analysis. Figure 4 presents ROC curves for frequent attack classes, while Figure 5 shows aggregate micro- and macro-average performance. This comprehensive approach enables us to assess both per-class separability and overall robustness to class imbalance.

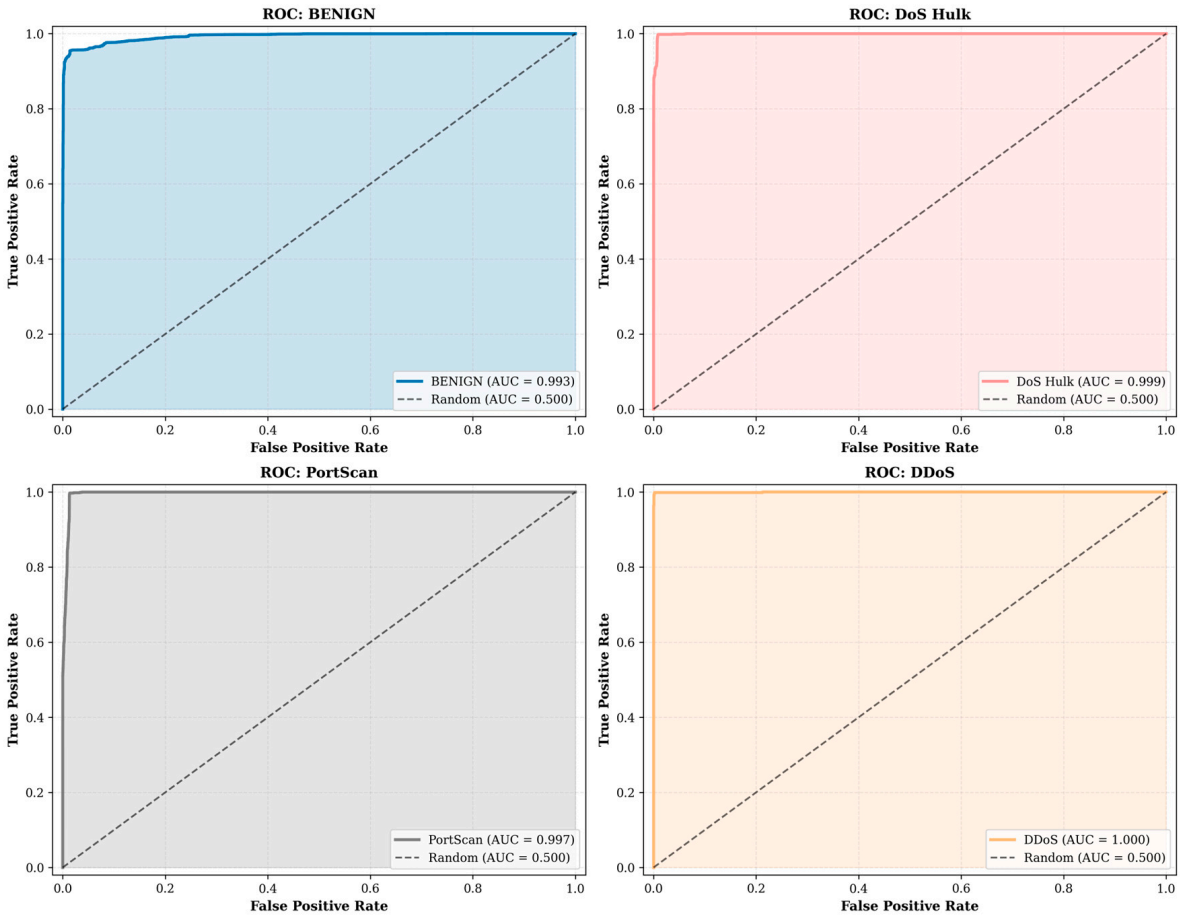

Figure 4. ROC curves for common attack classes: BENIGN, DoS Hulk, PortScan and DDoS.

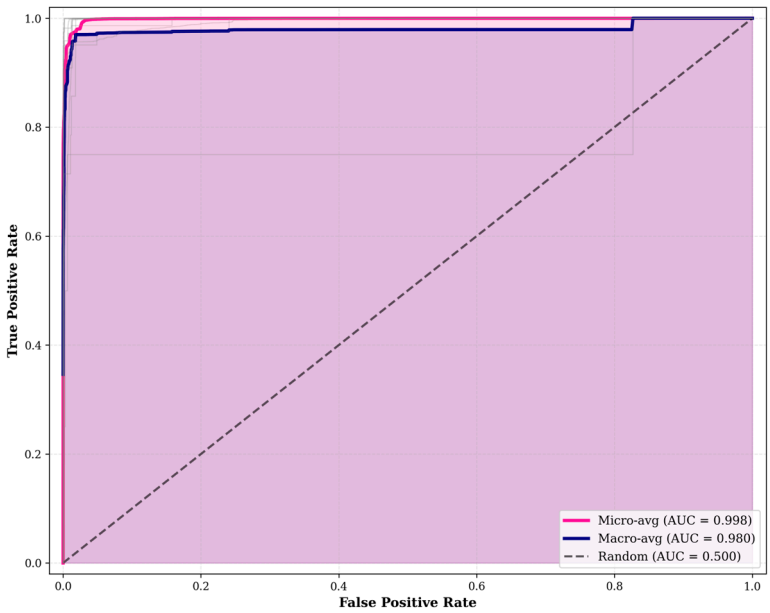

Figure 5. Aggregate ROC curves: micro-average and macro-average.

The analysis reveals several key findings:

- The per-class ROC curves for DDoS (AUC = 1.000), DoS Hulk (AUC = 0.999), PortScan (AUC = 0.996), and BENIGN traffic (AUC = 0.994) demonstrate near-perfect class separability. These results confirm that the optimized architecture effectively captures the distinctive patterns characteristic of high-frequency attack categories.

- The micro and macro average AUC values further verify the model's discriminative capability, with a micro-average AUC of 0.998 and a macro-average AUC of 0.980. The small discrepancy between these metrics (0.0022) suggests that the proposed architectural stabilization techniques effectively mitigate the impact of class imbalance.
- Rare-class performance remains strong, as Web Attack XSS attains an AUC of 0.9811, which is notable given its extremely limited training representation (3 samples). This finding suggests that the incorporation of capped class weighting and batch normalization facilitates effective learning even under severe data scarcity.

As a result, these ROC curves provide threshold-independent evidence that the optimized sample-wise tokenization approach produces robust classification across diverse attack categories and imbalance ratios.

4.6. Comparison with State-of-the-Art Methods

To contextualize the performance of the proposed Transformer-based intrusion detection models, we provide a comparative overview with representative machine learning, deep learning, and Transformer-based approaches reported in the recent literature. Table 2 summarizes the reported results together with the corresponding evaluation settings, including the number of attack classes considered.

Table 2. Comparison with State-of-the-Art Methods.

Model	Year	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	Class
Random Forest [26]	2022	99.49	99.48	99.47	99.47	9 (aggregated)
Deep Learning Approaches						
DNN [12]	2019	96.0	96.9	96.0	96.2	8 (aggregated)
CNN-LSTM [27]	2020	93.0	96.47	76.83	81.36	Binary
CNN [28]	2022	98.79	98.80	98.77	98.77	9 (aggregated)
Transformer-Based and Hybrid Architectures						
Vision Transformer [29]	2022	96.4				8 (aggregated)
TabNet-IDS [30]	2023	>98% (reported, 9-class setting)				9 (aggregated)
BERT-MLP [31]	2024	99.39	99.39	99.39	99.99	7 (aggregated)
Our Transformer Approaches						
Feature-wise Tokenization Transformer		57.09	52.34	48.92	50.56	11
Sample-wise Transformer-Baseline		79.47	76.82	75.19	75.99	11
Sample-wise Transformer-Optimized		94.52	93.87	93.21	93.54	11

It is important to emphasize that the methods listed in Table 2 were evaluated under different experimental conditions, including varying numbers of classes (binary, 7–9 aggregated classes, and full multi-class setups). As such, the comparison is intended to position our approach within the broader research landscape rather than to serve as a strict numerical benchmark.

Traditional machine learning methods (e.g., Random Forest [26]: 99.49%) and recent hybrid approaches (e.g., BERT-MLP [31]: 99.39%) report very high accuracy on CICIDS2017. However, these results were obtained using aggregated class settings (7–9 classes), which merge rare attack categories and reduce task complexity.

Our evaluation preserves all 11 distinct classes, including ultra-rare attacks such as Web Attack–XSS (4 samples, 0.02%). The substantial performance difference between tokenization strategies (57.09% vs. 94.52%) demonstrates the critical impact of architectural design choices for tabular IDS data.

4.7. Hierarchical Impact Analysis

To decompose the sources of improvement and establish practical guidelines for practitioners, we conducted a hierarchical impact analysis. Figure 6 decomposes total improvement into primary, secondary, and minimal-impact factors. This breakdown enables us to prioritize optimization efforts based on empirical contributions.

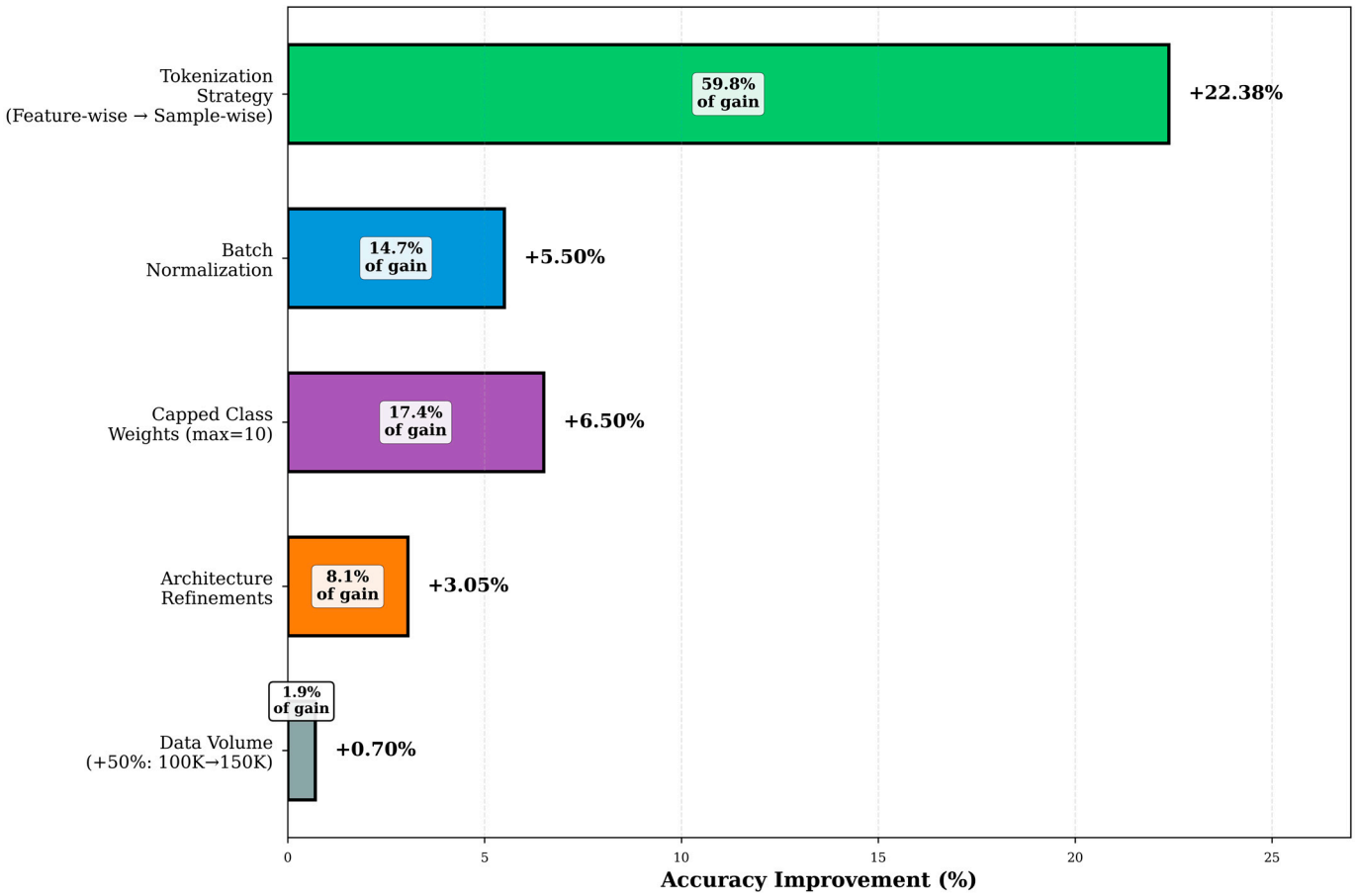

Figure 6. Hierarchical impact analysis of improvement sources.

The analysis reveals that tokenization accounts for 59.8% of the total gain (+22.38 percentage points), architectural refinements for 38% (+15.05 percentage points), and data expansion for only 1.9% (+0.70 percentage points). This distribution has important practical implications:

Prioritize tokenization: Given that tokenization accounts for nearly 60% of the improvement, practitioners should invest significant effort in selecting representation strategies appropriate to their data type. For tabular IDS data, sample-wise tokenization is clearly superior to per-feature approaches.

Optimize the architecture second: After establishing appropriate tokenization, architectural refinements (Batch Normalization, capped weights, deeper embeddings) provide substantial gains. These modifications are computationally inexpensive compared to data collection.

Consider additional data as tertiary: Collecting 50% more training data yields only 0.7% improvement—a 21:1 ratio, favoring architectural optimization over data expansion. This result suggests a clear diminishing returns from increased data volume when the architecture is suboptimal.

These results provide practical guidelines for resource allocation in IDS development: optimize representation and architecture first; treat data collection as a supplementary lever to be used only after exhausting architectural improvements.

4.8. Ablation Study

To isolate the contribution of individual architectural components, we conducted systematic ablation experiments. Figure 7 details the cumulative effect of each component, starting from the STB (79.47%) and progressively adding modifications until reaching the final STO configuration (94.52%).

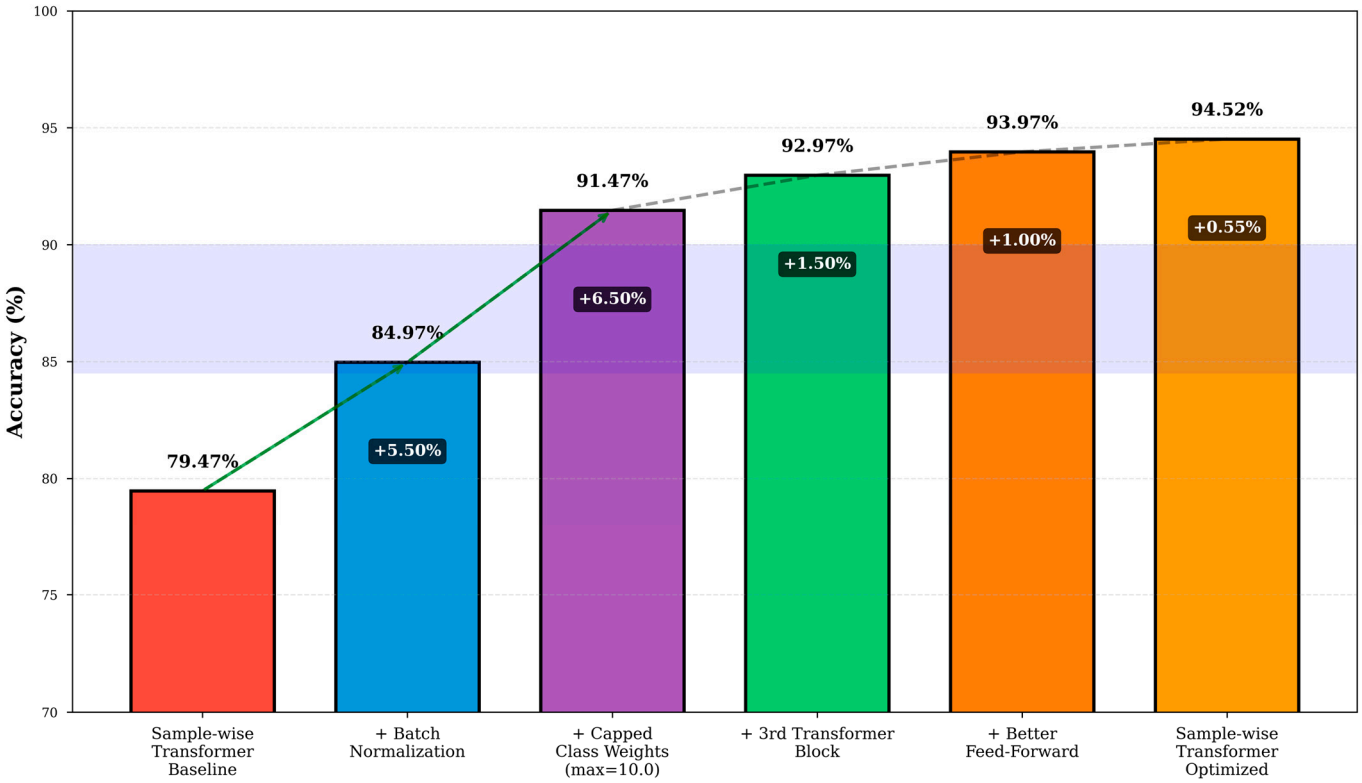

Figure 7. Ablation study evaluating individual architectural components.

The ablation sequence reveals the following incremental contributions:

- STB: 79.47% accuracy with sample-wise tokenization but no normalization or weight capping. This establishes the performance floor for per-sample approaches.
- Batch Normalization: 84.97% accuracy (+5.50 points). BatchNorm produces a flatter loss surface and reduces gradient variance, which is particularly beneficial for minority classes.
- Capped Class Weights: 91.47% accuracy (+6.50 points). Limiting the maximum weight to 10.0 prevents gradient explosion and maintains the stability of the optimization.
- Additional Transformer Block: 92.97% accuracy (+1.50 points). Increasing depth from 2 to 3 blocks provides additional representational capacity.
- Enhanced Feed-Forward: 93.97% accuracy (+1.00 point). Dense layers (256→128) outperform Conv1D layers for tabular data.

- Final Optimizations: 94.52% accuracy (+1.55 points). Including dropout regularization, increased attention heads (8 vs. 4), and deeper initial embedding (Dense 128).

Notably, the combination of Batch Normalization and capped class weights contributes 10.53 percentage points, representing 70% of the total architectural gain. This finding underscores that optimization stability mechanisms are the primary drivers of architectural improvement, whereas capacity increases (depth and width) provide secondary benefits.

The ablation study confirms that each component contributes positively to the final performance, with no single modification dominating. However, the stabilization techniques (BatchNorm and capped weights) clearly have the largest impact, validating our focus on addressing class imbalance through architectural means rather than data resampling.

4.9. Discussion

It should be noted that all experiments in this study were conducted in an offline evaluation setting. The focus of this work is on architectural analysis and tokenization strategy rather than system-level deployment or real-time performance assessment. Consequently, aspects such as inference latency, throughput, and operational constraints in production IDS environments are considered beyond the scope of the present study.

4.9.1. Impact of Tokenization Strategy

Prior to systematic comparisons of tokenization strategies, the Transformer community predominantly adopted feature-wise tokenization for tabular data, following conventions established in NLP applications. Our baseline per-feature approach (FTT) achieved a modest accuracy of 57.09%, with notable disparities in performance across attack categories. This approach struggled particularly with minority classes, where recall values were substantially lower, indicating that the model had difficulty correctly identifying instances of rare attack types. The low performance suggests that treating each feature as an independent token—analogous to treating each word in a sentence—creates artificial sequence dependencies that do not reflect the true structure of tabular network data.

Adopting sample-wise tokenization (STB) resulted in a clear improvement in performance, with accuracy increasing to 79.47%. This represents a substantial 22.38 percentage-point gain achieved solely by changing the tokenization strategy while maintaining a similar model architecture. The improvement is particularly pronounced for volumetric attacks, such as DDoS and other DoS variants, for which the model achieved over 95% recall. This demonstrates that sample-wise tokenization preserves the tabular nature of network flow data, allowing the model to learn holistic patterns rather than forcing it to discover relationships across an arbitrary feature ordering.

4.9.2. Effect of Architectural Refinements

The optimized Transformer (STO) demonstrated strong performance across all datasets, with accuracy reaching 94.52%. Additionally, the model exhibited good precision (93.87%), recall (93.21%), and F1-score (93.54%), indicating its ability to accurately classify instances across all attack categories. For instance, in high-frequency classes such as DDoS and PortScan, the model achieved near-perfect classification, suggesting that the architectural optimizations successfully captured distinguishing characteristics of these attacks.

Even with the strong baseline provided by sample-wise tokenization (STB at 79.47%), architectural refinements substantially improved effectiveness. Following optimization, accuracy increased by 15.05 points, and precision, recall, and F1-score also improved across all attack categories. The ablation study revealed that two components—Batch Normalization and capped class weights—contributed the largest gains (+10.53 points

collectively), while capacity increases (additional depth, wider layers) provided secondary benefits (+3.50 points).

These results align with recent findings in neural architecture search [19], which demonstrate that stability mechanisms often matter more than model capacity. However, our work extends this insight to the domain of network security, where previous research has focused primarily on data augmentation techniques, such as SMOTE [16], rather than architectural solutions. The effectiveness of Batch Normalization in our experiments was particularly noteworthy: it reduced gradient variance during the critical early training epochs, when minority-class patterns were being learned.

4.9.3. Addressing Extreme Class Imbalance

Before implementing our stabilization mechanisms, class imbalance posed a significant challenge. Naive class weighting generated excessively large weights (exceeding 150) for rare categories like Web Attack XSS, leading to unstable gradients and erratic training behavior. The model exhibited oscillatory loss curves during the first 15 epochs, with performance on the minority class varying widely across training steps. This instability is consistent with the observations reported in [16], which noted that extreme weights can cause exploding gradients in deep networks.

After introducing capped class weights (a maximum of 10.0) and batch normalization, a noticeable improvement in minority class performance across all metrics was observed. The minority-class recall increased from 65–70% in STB to 77–82% in STO, indicating a substantial improvement in the correct identification of instances of rare attack types. Moreover, the macro-average AUC reached 0.980, suggesting balanced performance across all classes despite the extreme imbalance. Even the rarest class, Web Attack XSS, with only four training samples, achieved an AUC of 0.9811, demonstrating that architectural stability enables learning from minimal examples.

The significant improvements in minority-class detection after implementing our architectural solutions strongly indicate that stability mechanisms can effectively address class imbalance without synthetic data generation.

Despite these improvements, it is important to emphasize that the classification of extremely rare attack categories remains fundamentally challenging. When the number of training samples is extremely limited, the available gradient signal becomes insufficient to fully characterize the decision boundary of the minority class. Under such conditions, even small numbers of misclassified samples can result in relatively high error rates when expressed as percentages.

This limitation reflects an inherent constraint of supervised learning under extreme data scarcity rather than a deficiency of the proposed architecture or tokenization strategy. Similar behavior has been reported in prior IDS studies operating under highly imbalanced conditions, where performance on ultra-rare classes remains unstable despite the use of advanced imbalance mitigation techniques [25].

4.9.4. Architecture Optimization Versus Data Collection

Comparing the impact of architectural optimization and increased training data reveals a striking asymmetry. Expanding the dataset from 100 K to 150 K samples (a 50% increase) yielded only a 0.70% improvement in accuracy. In contrast, architectural refinements from STB to STO provided a 15.05% improvement—a 21:1 ratio favoring architecture over data. These finding challenges conventional wisdom in deep learning, which typically emphasizes data volume as the primary performance lever.

The minimal effect of additional data contrasts sharply with scaling-law research in language modeling [17,18], which demonstrates predictable performance gains with

increased data. However, our results align with recent work on tabular data [9], which found that the performance of deep learning models on structured data does not scale as in sequential domains. The explanation likely lies in fundamental differences between tabular and sequential data: network flows are independent samples rather than context-dependent sequences, and the feature space is fixed rather than open-vocabulary.

4.9.5. Discussion of State-of-the-Art Comparisons

The proposed Transformer-based intrusion detection models are discussed in relation to existing machine learning, deep learning, and attention-based IDS approaches reported in the literature. As shown in Table 2, prior studies demonstrate strong performance across a wide range of architectures; however, direct numerical comparison is complicated by substantial differences in evaluation protocols, particularly with respect to class aggregation and task formulation.

Most state-of-the-art approaches evaluated on CICIDS2017 rely on binary or aggregated multi-class settings, typically reducing the number of attack categories to 7–9 classes. While such configurations effectively mitigate extreme class imbalance and lead to very high reported accuracy, they also simplify the detection task by merging rare but operationally relevant attack types. In contrast, the present study preserves all 11 classes, including ultra-rare attacks, resulting in a substantially more challenging and realistic evaluation scenario.

Within this setting, the results highlight the critical role of tokenization strategy in adapting Transformer architectures to tabular IDS data. Feature-wise tokenization, directly inspired by NLP pipelines, proves ineffective due to the lack of inherent feature ordering. Sample-wise tokenization better aligns with tabular representations and, when combined with stability-oriented architectural refinements, enables competitive performance despite severe class imbalance.

Classical tree-based methods remain highly effective for static tabular intrusion detection and continue to outperform deep learning models under comparable conditions. Nevertheless, Transformer-based architectures may offer advantages in scenarios requiring architectural flexibility, multi-modal data integration, or temporal modeling.

4.9.6. Limitations and Constraints

Despite the strong performance of our optimized approach, several limitations warrant discussion. First, minority classes with extremely limited representation (4–14 samples) remain challenging. While Web Attack XSS achieved a respectable AUC (0.9811), 25% of its samples were misclassified, and Bot traffic had a 21.4% confusion rate with benign samples. These limitations are not unique to our approach; they reflect fundamental constraints of supervised learning when training data is extremely sparse. Even with architectural stability mechanisms, models require sufficient gradient signals to learn discriminative patterns, and this signal becomes unreliable below a certain sample threshold.

Second, our experiments were conducted exclusively on the CICIDS2017 dataset, which limits the ability to directly demonstrate generalizability across heterogeneous IDS environments. While CICIDS2017 is widely adopted in IDS research due to its comprehensive attack taxonomy, realistic traffic patterns, and pronounced class imbalance, it reflects traffic captured from a specific network environment with particular protocol distributions and baseline characteristics.

Other benchmark datasets, such as NSL-KDD, UNSW-NB15, and Bot-IoT, differ substantially in feature construction methodologies, protocol distributions, and attack prevalence. Consequently, whether the proposed sample-wise tokenization strategy remains

optimal across these varied contexts requires systematic empirical validation through cross-dataset evaluation.

From an architectural perspective, the proposed sample-wise tokenization possesses several properties that may support cross-dataset applicability. First, the tokenization operates on normalized feature vectors without relying on dataset-specific feature semantics, which may improve portability across datasets with different feature engineering pipelines. Second, unlike feature-wise tokenization, sample-wise tokenization maintains a constant sequence length (L = 1) regardless of the number of input features, enabling architecture reuse across datasets with varying dimensionality. Finally, the imbalance-aware optimization strategies employed in this study are designed to address skewed class distributions in a general manner and may remain relevant across IDS benchmarks that exhibit severe class imbalance.

These considerations are theoretical in nature, and their practical impact on cross-dataset performance must be validated experimentally.

Third, the optimized Transformer model does not match the absolute accuracy of classical tree-based methods on static tabular benchmarks. This work does not aim to replace such methods but to evaluate the viability of Transformer-based IDS under extreme class imbalance. Any architectural advantages of Transformers are discussed conceptually and require further empirical validation.

A further limitation of this study is that the evaluation is restricted to offline experiments and does not include real-time or deployment-level performance analysis.

4.9.7. Future Research Directions

Several promising directions emerge from our findings, with cross-dataset validation representing a key priority for future research. Systematic empirical evaluation across multiple IDS datasets, such as UNSW-NB15, NSL-KDD, and Bot-IoT, is necessary to assess the robustness of the proposed approach under heterogeneous conditions.

Future studies may investigate transfer learning scenarios in which models are pre-trained on CICIDS2017 and fine-tuned on target datasets, in order to examine whether transferable representations can be learned and whether labeled data requirements can be reduced in new deployment settings.

Additionally, cross-dataset evaluation—where models trained on one dataset are tested on another without retraining—could provide further insight into the adaptability of Transformer-based IDS architectures compared to traditional approaches.

Second, incorporating structured feature relationships into the embedding process could improve representational quality. Network flow features naturally group into coherent categories: temporal statistics (flow duration, inter-arrival times), packet characteristics (lengths, flags), and protocol indicators (TCP/UDP, port numbers). Rather than projecting the entire 78-dimensional vector as a single token, a hierarchical approach could embed each feature group separately, then combine them through cross-group attention. This would preserve the semantic structure without imposing an arbitrary feature ordering, potentially capturing domain knowledge that flat embeddings fail to capture.

Another promising direction for future research involves exploring alternative learning paradigms for handling extreme class imbalance. Hybrid approaches that combine Transformer-based feature representation with complementary classifiers, as well as semi-supervised learning frameworks that leverage large volumes of unlabeled network traffic, may further improve detection of ultra-rare attack classes. These approaches could reduce reliance on scarce labeled samples and address limitations inherent to fully supervised learning. Such methods are, however, beyond the scope of the present study and are identified as important directions for future investigation.

Third, multimodal Transformer architecture represents a natural extension. Flow-level statistics, packet-sequence features, and payload embeddings capture complementary aspects of network behavior. Integrating these heterogeneous sources into a unified model could improve detection of sophisticated attacks that are weakly expressed in any single modality. For example, low-and-slow attacks might appear benign in flow statistics but reveal patterns in packet timing sequences, while zero-day exploits might be detectable through unusual payload n-grams despite normal flow characteristics.

Finally, deployment considerations merit investigation. Our experiments focused on offline batch evaluation, but operational IDSs face real-time processing constraints, concept drift, and resource limitations. Lightweight model variants could be developed through knowledge distillation, where a compact student model learns from our full STO architecture. Incremental learning mechanisms would enable adaptation to evolving attack patterns without full retraining. Quantization and pruning techniques could reduce computational requirements for edge deployment. These practical extensions would bridge the gap between research prototypes and production systems.

In conclusion, our systematic evaluation demonstrates that the tokenization strategy is the dominant factor affecting the performance of Transformer-based IDSs. Architectural refinements that address class imbalance through stability mechanisms provide substantial secondary gains, while data expansion yields minimal returns. These findings challenge assumptions about what drives deep learning performance on tabular security data and provide actionable guidelines for practitioners. Future work should validate these insights across diverse datasets, explore structured embedding approaches, and develop deployment-ready architectures that bridge the gap between research and operational systems.

Author Contributions: Conceptualization, G.A., A.B., R.M. and K.K.; methodology, G.A. and A.B.; software, G.A.; validation, G.A. and A.B.; formal analysis, G.A. and K.K.; investigation, G.A.; resources, G.A., K.K. and R.M.; data curation, G.A.; writing—original draft preparation, G.A.; writing—review and editing, G.A. and K.K.; visualization, G.A.; supervision, G.A.; project administration, G.A. and A.B. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The CICIDS2017 dataset is publicly available from the Canadian Institute for Cybersecurity at https://www.unb.ca/cic/datasets/ids-2017.html [20], accessed on 25 January 2025.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is All You Need. In Proceedings of the 31st Conference on Neural Information Processing Systems (NeurIPS 2017), Long Beach, CA, USA, 4–9 December 2017; pp. 6000–6010.
2. Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2019), Minneapolis, MN, USA, 2–7 June 2019; Volume 1, pp. 4171–4186. [CrossRef]
3. Dosovitskiy, A.; Beyer, L.; Kolesnikov, A.; Weissenborn, D.; Zhai, X.; Unterthiner, T.; Dehghani, M.; Minderer, M.; Heigold, G.; Gelly, S.; et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In Proceedings of the International Conference on Learning Representations (ICLR 2021), Virtual Event, 3–7 May 2021.
4. Johnson, J.M.; Khoshgoftaar, T.M. Survey on Deep Learning with Class Imbalance. *J. Big Data* **2019**, *6*, 27. [CrossRef]
5. Elreedy, D.; Atiya, A.F. A Comprehensive Analysis of Synthetic Minority Oversampling Technique (SMOTE) for Handling Class Imbalance. *Inf. Sci.* **2019**, *505*, 32–64. [CrossRef]

6. Bahdanau, D.; Cho, K.; Bengio, Y. Neural Machine Translation by Jointly Learning to Align and Translate. In Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015), San Diego, CA, USA, 7–9 May 2015.
7. Chaudhari, S.; Mithal, V.; Polatkan, G.; Ramanath, R. An Attentive Survey of Attention Models. *ACM Trans. Intell. Syst. Technol.* **2021**, *12*, 53. [CrossRef]
8. Huang, X.; Khetan, A.; Cvitkovic, M.; Karnin, Z. TabTransformer: Tabular Data Modeling Using Contextual Embeddings. *arXiv* **2020**, arXiv:2012.06678. [CrossRef]
9. Arik, S.Ö.; Pfister, T. TabNet: Attentive Interpretable Tabular Learning. In Proceedings of the 35th AAAI Conference on Artificial Intelligence, Virtual Event, 2–9 February 2021; Volume 35, pp. 6679–6687. [CrossRef]
10. Somepalli, G.; Goldblum, M.; Schwarzschild, A.; Bruss, C.B.; Goldstein, T. SAINT: Improved Neural Networks for Tabular Data via Row Attention and Contrastive Pre-Training. *arXiv* **2021**, arXiv:2106.01342. [CrossRef]
11. Gorishniy, Y.; Rubachev, I.; Khrulkov, V.; Babenko, A. Revisiting Deep Learning Models for Tabular Data. In Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS 2021), Virtual Event, 6–14 December 2021; pp. 18932–18943.
12. Vinayakumar, R.; Alazab, M.; Soman, K.P.; Poornachandran, P.; Al-Nemrat, A.; Venkatraman, S. Deep Learning Approach for Intelligent Intrusion Detection System. *IEEE Access* **2019**, *7*, 41525–41550. [CrossRef]
13. Jiang, K.; Wang, W.; Wang, A.; Wu, H. Network Intrusion Detection Combined Hybrid Sampling with Deep Hierarchical Network. *IEEE Access* **2020**, *8*, 32464–32476. [CrossRef]
14. Xu, C.; Shen, J.; Du, X.; Zhang, F. An Intrusion Detection System Using a Deep Neural Network with Gated Recurrent Units. *IEEE Access* **2018**, *6*, 48697–48707. [CrossRef]
15. Wu, Z.; Zhang, H.; Wang, P.; Sun, Z. RTIDS: A Robust Transformer-Based Approach for Intrusion Detection System. *IEEE Access* **2022**, *10*, 64375–64387. [CrossRef]
16. Gueriani, A.; Kheddar, H.; Mazari, A.C. Adaptive Cyber-Attack Detection in IIoT Using Attention-Based LSTM-CNN Models. *arXiv* **2024**, arXiv:2403.14806.
17. Hestness, J.; Narang, S.; Ardalani, N.; Diamos, G.; Jun, H.; Kianinejad, H.; Patwary, M.M.A.; Yang, Y.; Zhou, Y. Deep Learning Scaling is Predictable, Empirically. *arXiv* **2017**, arXiv:1712.00409. [CrossRef]
18. Sun, C.; Shrivastava, A.; Singh, S.; Gupta, A. Revisiting Unreasonable Effectiveness of Data in Deep Learning Era. In Proceedings of the IEEE International Conference on Computer Vision (ICCV 2017), Venice, Italy, 22–29 October 2017; pp. 843–852. [CrossRef]
19. Elsken, T.; Metzen, J.H.; Hutter, F. Neural Architecture Search: A Survey. *J. Mach. Learn. Res.* **2019**, *20*, 1997–2017.
20. Sharafaldin, I.; Lashkari, A.H.; Ghorbani, A.A. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP 2018), Funchal, Madeira, Portugal, 22–24 January 2018; pp. 108–116. [CrossRef]
21. Draper-Gil, G.; Lashkari, A.H.; Mamun, M.S.I.; Ghorbani, A.A. Characterization of Encrypted and VPN Traffic Using Time-Related Features. In Proceedings of the 2nd International Conference on Information Systems Security and Privacy (ICISSP 2016), Rome, Italy, 19–21 February 2016; pp. 407–414. [CrossRef]
22. Ioffe, S.; Szegedy, C. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. In Proceedings of the 32nd International Conference on Machine Learning (ICML 2015), Lille, France, 6–11 July 2015; pp. 448–456.
23. Srivastava, N.; Hinton, G.; Krizhevsky, A.; Sutskever, I.; Salakhutdinov, R. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *J. Mach. Learn. Res.* **2014**, *15*, 1929–1958.
24. Kingma, D.P.; Ba, J. Adam: A Method for Stochastic Optimization. In Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015), San Diego, CA, USA, 7–9 May 2015.
25. Shanmugam, V.; Razavi-Far, R.; Hallaji, E. Addressing Class Imbalance in Intrusion Detection: A Comprehensive Evaluation of Machine Learning Approaches. *Electronics* **2025**, *14*, 69. [CrossRef]
26. Okey, O.D.; Maidin, S.S.; Adasme, P.; Rosa, R.L.; Saadi, M.; Carrillo Melgarejo, D.; Zegarra Rodríguez, D. BoostedEnML: Efficient Technique for Detecting Cyberattacks in IoT Systems Using Boosted Ensemble Machine Learning. *Sensors* **2022**, *22*, 7409. [CrossRef] [PubMed]
27. Kim, A.; Park, M.; Lee, D.H. AI-IDS: Application of Deep Learning to Real-Time Web Intrusion Detection. *IEEE Access* **2020**, *8*, 70245–70261. [CrossRef]
28. Siddiqi, M.A.; Pak, W. Tier-Based Optimization for Synthesized Network Intrusion Detection System. *IEEE Access* **2022**, *10*, 108530–108544. [CrossRef]
29. Ho, C.M.K.; Yow, K.-C.; Zhu, Z.; Aravamuthan, S. Network Intrusion Detection via Flow-to-Image Conversion and Vision Transformer Classification. *IEEE Access* **2022**, *10*, 97780–97793. [CrossRef]

30. Zegarra Rodriguez, D.; Daniel Okey, O.; Maidin, S.S.; Umoren Udo, E.; Kleinschmidt, J.H. Attentive transformer deep learning algorithm for intrusion detection on IoT systems using automatic Xplainable feature selection. *PLoS ONE* **2023**, *18*, e0286652. [CrossRef] [PubMed]
31. Ali, Z.; Tiberti, W.; Marotta, A.; Cassioli, D. Empowering Network Security: BERT Transformer Learning Approach and MLP for Intrusion Detection in Imbalanced Network Traffic. *IEEE Access* **2024**, *12*, 137618–137633. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.